

HormuzNet — Análise Completa do Código

Foco: Concorrência e Conectividade em Redes

Gerado em: 2026-05-13 22:42:08 (America/Bahia)

Workspace: /home/luis/Área de trabalho/HormuzNet

1. Visão geral do sistema

HormuzNet simula um ecossistema distribuído para monitoramento marítimo e resposta a ocorrências em tempo quase-real. A arquitetura é descentralizada: múltiplos Brokers formam uma malha P2P, recebendo leituras de Sensores via UDP multicast. As leituras viram Ocorrências que entram em uma fila de prioridade. Drones (agentes autônomos) mantêm conexões TCP com brokers para receber comandos e enviar telemetria/estado. Um Monitor web observa os brokers por TCP e expõe um dashboard HTTP com atualização via WebSocket.

2. Arquitetura

[Sensores] -- UDP Multicast (224.0.0.1:8080) --> [Brokers B1..B4]

[Brokers] formam malha P2P via TCP (Lamport + heartbeat + sincronização de drones)

[Drones] <==== TCP persistente + fallback ==== [Broker primário / vizinhos]

[Monitor] -- TCP observer --> [Brokers]

[Navegador] -- HTTP/WS --> [Monitor]

3. Estrutura de arquivos

Arquivo	Tipo	Papel
.gitignore	Git config	Ignora artefatos locais.
README.md	Markdown	Descrição do projeto e instruções de execução com Docker Compose.
code/Dockerfile.broker	Dockerfile	Build multi-stage do binário do broker (scratch).
code/Dockerfile.drone	Dockerfile	Build multi-stage do binário do drone

		(entrypoint: base_exec).
code/Dockerfile.monitor	Dockerfile	Build do monitor + assets estáticos (mapa).
code/Dockerfile.sensor	Dockerfile	Build multi-stage do binário do sensor (scratch).
code/assets/map.png	Imagem	Asset do dashboard (mapa de fundo).
code/cmd/broker/ broker_main.go	Go	Processo Broker: recebe UDP, aceita TCP (drones/brokers), sincroniza malha e despacha drones.
code/cmd/drone/ drone_main.go	Go	Processo Drone: conecta via TCP ao broker (fallback), recebe comandos e executa missão.
code/cmd/monitor/ monitor_main.go	Go	Processo Monitor: TCP observer nos brokers + dashboard HTTP/WebSocket.
code/cmd/sensor/ sensor_main.go	Go	Processo Sensor: gera leituras e publica por UDP multicast periodicamente.
code/ generate_compose.py	Python	Gera docker-compose.yml com sensores aleatórios por setor (script de apoio).
code/go.mod	Go module	Módulo Go (Go 1.23).
code/internal/fila/fila.go	Go	Fila de prioridade thread-

		safe para ocorrências (heap + mutex).
code/internal/models/dados.go	Go	Modelos de dados e mensagens (sensores, ocorrências, drones, comandos).
docker-compose.yml	YAML	Orquestração do cluster: 4 brokers, 6 drones, 1 monitor e sensores (containers).
docs/HormuzNet_Analise_Conc orrencia_Conectividade.docx	Outro	—
docs/arquitetura_descentraliza da.md	Markdown	Documento de arquitetura (Lamport, P2P, UDP multicast, exclusão mútua).
start.sh	Shell	Script para rodar localmente (go run) em múltiplos terminais; útil fora do Docker.
stop.sh	Shell	Script para encerrar processos iniciados via go run/binários locais.

4. Análise arquivo a arquivo

Abaixo: (a) resumo do papel do arquivo, (b) função a função quando aplicável, e (c) pontos de concorrência/rede por varredura de padrões.

.gitignore

Ignora artefatos locais.

Concorrência/Redes — pontos de interesse (scan)

Categoria	Linha	Trecho
-----------	-------	--------

Sem hits por scan automático.

README.md

Descrição do projeto e instruções de execução com Docker Compose.

Concorrência/Redes — pontos de interesse (scan)

Categoria	Linha	Trecho
-----------	-------	--------

Sem hits por scan automático.

code/Dockerfile.broker

Build multi-stage do binário do broker (scratch).

Concorrência/Redes — pontos de interesse (scan)

Categoria	Linha	Trecho
goroutine	4	RUN go mod download
goroutine	6	RUN CGO_ENABLED=0 go build -o broker_exec ./cmd/broker/

code/Dockerfile.drone

Build multi-stage do binário do drone (entrypoint: base_exec).

Concorrência/Redes — pontos de interesse (scan)

Categoria	Linha	Trecho
goroutine	4	RUN go mod download
goroutine	6	RUN CGO_ENABLED=0 go build -o base_exec

`./cmd/drone/`

code/Dockerfile.monitor

Build do monitor + assets estáticos (mapa).

Concorrência/Redes — pontos de interesse (scan)

Categoria	Linha	Trecho
goroutine	4	RUN go mod download
goroutine	6	RUN CGO_ENABLED=0 go build -o monitor_exec ./cmd/monitor/

code/Dockerfile.sensor

Build multi-stage do binário do sensor (scratch).

Concorrência/Redes — pontos de interesse (scan)

Categoria	Linha	Trecho
goroutine	4	RUN go mod download
goroutine	6	RUN CGO_ENABLED=0 go build -o sensor_exec ./cmd/sensor/

code/assets/map.png

Asset do dashboard (mapa de fundo).

code/cmd/broker/broker_main.go

Processo Broker: recebe UDP, aceita TCP (drones/brokers), sincroniza malha e despacha drones.

Função a função

novoBroker — code/cmd/broker/broker_main.go:62

Assinatura: func novoBroker(id, setorID, portaUDP, portaTCP string) *Broker {

O que faz: Inicializa o Broker (IDs, portas, relógio Lamport, fila, mapas e logger).

tick — code/cmd/broker/broker_main.go:80

Assinatura: func (b *Broker) tick() int {

O que faz: Incrementa o relógio lógico de Lamport (seção crítica com mutex) e retorna o novo valor.

syncLamport — code/cmd/broker/broker_main.go:87

Assinatura: func (b *Broker) syncLamport(recebido int) {

O que faz: Atualiza o relógio local ao receber Lamport remoto: $b.lamport = \max(b.lamport, recebido) + 1$.

main — code/cmd/broker/broker_main.go:98

Assinatura: func main() {

O que faz: Parseia flags, cria broker e inicia as goroutines de rede (TCP/UDP), P2P (vizinhos), e loops periódicos (HB, falhas, envelhecimento, despacho, ociosidade).

escutarUDP — code/cmd/broker/broker_main.go:130

Assinatura: func (b *Broker) escutarUDP() {

O que faz: Entra no grupo UDP multicast e fica lendo datagramas; para cada leitura, cria goroutine de processamento.

processarLeitura — code/cmd/broker/broker_main.go:152

Assinatura: func (b *Broker) processarLeitura(dados []byte) {

O que faz: Decodifica JSON do sensor, valida criticidade, gera Ocorrencia com LamportTime e enfileira na FilaPrioridade.

escutarTCP — code/cmd/broker/broker_main.go:184

Assinatura: func (b *Broker) escutarTCP() {

O que faz: Abre listener TCP para aceitar conexões (drones e brokers). Cada Accept gera goroutine de identificação.

identificarConexao — code/cmd/broker/broker_main.go:201

Assinatura: func (b *Broker) identificarConexao(conn net.Conn) {

O que faz: Lê primeira mensagem JSON para classificar conexão como Drone (REGISTRO_DRONE) ou Broker (REGISTRO). Encaminha para o loop de leitura apropriado e dispara sincronização global (SINC_DRONE).

registrarDroneLocal — code/cmd/broker/broker_main.go:233

Assinatura: func (b *Broker) registrarDroneLocal(droneID string, info *models.InfoDrone, conn net.Conn) {

O que faz: Registra conexão TCP do drone no mapa local e atualiza a lista global de drones; broadcast SINC_DRONE para vizinhos.

loopLeituraDrone — code/cmd/broker/broker_main.go:258

Assinatura: func (b *Broker) loopLeituraDrone(droneID string, conn net.Conn, scanner *bufio.Scanner) {

O que faz: Loop de leitura das mensagens do drone (Scanner). No defer, trata desconexão como perda: marca ABATIDO, broadcast DRONE_PERDIDO e chama tratarDronePerdido.

processarMensagemDrone — code/cmd/broker/broker_main.go:298

Assinatura: func (b *Broker) processarMensagemDrone(msg models.MensagemDrone) {

O que faz: Processa KEEPALIVE/DRONE_ESTADO: atualiza telemetria/estado, broadcast SINC_DRONE e notifica MISSAO_CONCLUIDA quando aplicável.

tratarDronePerdido — code/cmd/broker/broker_main.go:354

Assinatura: func (b *Broker) tratarDronePerdido(d models.InfoDrone) {

O que faz: Trata perda/abate em missão: “desmarca” ocorrência como atendida para permitir re-despacho (simplificação; sem histórico da ocorrência).

droneMaisProximo — code/cmd/broker/broker_main.go:371

Assinatura: func (b *Broker) droneMaisProximo(oc models.Ocorrencia) (models.InfoDrone, bool) {

O que faz: Seleciona drone disponível com menor distância (ao quadrado); em empate, maior bateria.

marcarOcupado — code/cmd/broker/broker_main.go:393

Assinatura: func (b *Broker) marcarOcupado(droneID, ocorrenciaID string) {

O que faz: Atualiza estado do drone no mapa global para DESPACHADO, vinculando ocorrenciaID.

loopDespachar — code/cmd/broker/broker_main.go:407

Assinatura: func (b *Broker) loopDespachar() {

O que faz: Ticker periódico chamando tentarDespachar (política de despacho reativa).

tentarDespachar — code/cmd/broker/broker_main.go:415

Assinatura: func (b *Broker) tentarDespachar() {

O que faz: Olha topo da fila, checa mapa atendidos, escolhe drone. Se o drone está conectado LOCALMENTE, envia ComandoDrone DESPACHAR_DRONE, marca atendido e faz broadcast DRONE_DESPACHADO (exclusão mútua simplificada).

loopVerificarOciosidade — code/cmd/broker/broker_main.go:479

Assinatura: func (b *Broker) loopVerificarOciosidade() {

O que faz: Ticker que loga drones disponíveis há muito tempo sem despacho (observabilidade).

conectarVizinhos — code/cmd/broker/broker_main.go:501

Assinatura: func (b *Broker) conectarVizinhos(enderecos string) {

O que faz: Divide lista CSV de vizinhos e cria goroutine de conexão para cada um.

conectarVizinho — code/cmd/broker/broker_main.go:507

Assinatura: func (b *Broker) conectarVizinho(addr string) {

O que faz: Mantém conexão TCP com vizinho com backoff: envia MsgRegistro, lê mensagens, reidentifica idReal quando recebe BrokerID, e reconecta quando cai.

registrarVizinho — code/cmd/broker/broker_main.go:566

Assinatura: func (b *Broker) registrarVizinho(brokerID string, conn net.Conn) {

O que faz: Registra vizinho aceito passivamente (Accept) e inicializa último heartbeat.

loopLeituraBroker — code/cmd/broker/broker_main.go:576

Assinatura: func (b *Broker) loopLeituraBroker(brokerID string, conn net.Conn, scanner *bufio.Scanner) {

O que faz: Loop de leitura de mensagens do vizinho; no defer, remove vizinho do mapa.

processarMensagemBroker — code/cmd/broker/broker_main.go:593

Assinatura: func (b *Broker) processarMensagemBroker(msg models.MensagemBroker) {

O que faz: Atualiza Lamport + heartbeat e processa mensagens: SINC_DRONE, REQUISICAO_DRONE (enfileira), DRONE_DESPACHADO (marca atendido/remover), DRONE_PERDIDO (marca abatido).

enviarSincGlobal — code/cmd/broker/broker_main.go:651

Assinatura: func (b *Broker) enviarSincGlobal(conn net.Conn) {

O que faz: Ao registrar um peer, envia o estado atual de drones (SINC_DRONE) para acelerar convergência.

loopHeartbeat — code/cmd/broker/broker_main.go:675

Assinatura: func (b *Broker) loopHeartbeat() {

O que faz: Ticker que faz broadcast MsgHeartbeat para vizinhos.

loopDetectarFalhas — code/cmd/broker/broker_main.go:688

Assinatura: func (b *Broker) loopDetectarFalhas() {

O que faz: Ticker que invalida vizinhos sem heartbeat há heartbeatTimeout (detecção de falhas).

loopEnvelhecerFila — code/cmd/broker/broker_main.go:705

Assinatura: func (b *Broker) loopEnvelhecerFila() {

O que faz: Ticker que chama fp.Envelhecer para evitar starvation na fila.

broadcastVizinhos — code/cmd/broker/broker_main.go:715

Assinatura: func (b *Broker) broadcastVizinhos(msg models.MensagemBroker) {

O que faz: Envia mensagem JSON a todos os vizinhos em goroutines (paraleliza escrita; sem backpressure).

splitCSV — code/cmd/broker/broker_main.go:731

Assinatura: func splitCSV(s string) []string {

O que faz: Utilitário para separar lista CSV simples (sem suporte a espaços/escape).

Concorrência/Redes — pontos de interesse (scan)

Categoria	Linha	Trecho
goroutine	114	go b.escutarTCP()
goroutine	115	go b.escutarUDP()
goroutine	117	go b.conectarVizinhos(*vizSt r)
goroutine	119	go b.loopHeartbeat()
goroutine	120	go b.loopDetectarFalhas()
goroutine	121	go b.loopEnvelhecerFila()
goroutine	122	go b.loopDespachar()
goroutine	123	go b.loopVerificarOciosidade ()
goroutine	148	go b.processarLeitura(buf[:n)
goroutine	197	go b.identificarConexao(con n)
goroutine	214	go

		b.loopLeituraDrone(md.DroneID, conn, scanner)
goroutine	223	go b.loopLeituraBroker(msg.BrokerID, conn, scanner)
goroutine	503	go b.conectarVizinho(addr)
goroutine	720	go func() {
ticker	408	ticker := time.NewTicker(despachoInterval)
ticker	480	ticker := time.NewTicker(30 * time.Second)
ticker	676	ticker := time.NewTicker(heartbeatInterval)
ticker	689	ticker := time.NewTicker(heartbeatInterval)
ticker	706	ticker := time.NewTicker(envelhecerInterval)
mutex	37	lamportMu sync.Mutex
mutex	42	dronesLocaisMu sync.RWMutex
mutex	46	dronesMu sync.RWMutex
mutex	50	vizinhosMu sync.RWMutex
mutex	53	heartbeatMu sync.Mutex

mutex	56	atendidosMu sync.Mutex
tcp_listen	185	ln, err := net.Listen("tcp", b.portaTCP)
tcp_dial	510	conn, err := net.DialTimeout("tcp", addr, 5*time.Second)
udp_multicast	135	conn, err := net.ListenMulticastUDP(" udp", nil, addr)
json	154	if err := json.Unmarshal(dados, &leitura); err != nil {
json	212	if err := json.Unmarshal(linha, &md); err == nil && md.Tipo == models.DroneRegistro {
json	219	if err := json.Unmarshal(linha, &msg); err == nil && msg.Tipo == models.MsgRegistro {
json	291	if err := json.Unmarshal(scanner. Bytes(), &msg); err != nil {
json	454	json.NewEncoder(conn).E ncode(cmd)
json	526	if err := json.NewEncoder(conn).E ncode(reg); err != nil {

json	543	if err := json.Unmarshal(scanner. Bytes(), &msg); err != nil {
json	586	if err := json.Unmarshal(scanner. Bytes(), &msg); err != nil {
json	669	json.NewEncoder(conn).E ncode(msg)
json	722	if err := json.NewEncoder(conn).E ncode(msg); err != nil {

code/cmd/drone/drone_main.go

Processo Drone: conecta via TCP ao broker (fallback), recebe comandos e executa missão.

Função a função

novoDrone — code/cmd/drone/drone_main.go:41

Assinatura: func novoDrone(id string, addrs []string, posX, posY float64) *Drone {

O que faz: Inicializa drone com estado DISPONIVEL, posição inicial e logger; define lista de brokers para fallback.

main — code/cmd/drone/drone_main.go:57

Assinatura: func main() {

O que faz: Parseia flags (id, brokers, posição) e entra no loopConexao (bloqueante).

loopConexao — code/cmd/drone/drone_main.go:82

Assinatura: func (d *Drone) loopConexao() {

O que faz: Mantém conexão TCP com brokers em ordem de prioridade com backoff exponencial. Ao conectar: envia registro, inicia keepalive em goroutine e entra no loop de comandos; ao cair: fecha e tenta próximo broker.

enviarRegistro — code/cmd/drone/drone_main.go:137

Assinatura: func (d *Drone) enviarRegistro() error {

O que faz: Envia MensagemDrone REGISTRO_DRONE (JSON) com snapshot do InfoDrone no início da conexão.

loopKeepalive — code/cmd/drone/drone_main.go:158

Assinatura: func (d *Drone) loopKeepalive(stop <-chan struct{}) {

O que faz: Ticker que envia KEEPALIVE_DRONE periodicamente com snapshot do estado; roda em goroutine até stop ser fechado.

processarComandos — code/cmd/drone/drone_main.go:188

Assinatura: func (d *Drone) processarComandos(conn net.Conn) {

O que faz: Lê ComandoDrone do broker via Scanner (JSON por linha) e dispara execução em goroutine por comando.

executarComando — code/cmd/drone/drone_main.go:199

Assinatura: func (d *Drone) executarComando(cmd models.ComandoDrone) {

O que faz: Roteia o comando: DESPACHAR_DRONE → iniciarMissao; RETORNAR_DRONE → abortarMissao.

iniciarMissao — code/cmd/drone/drone_main.go:210

Assinatura: func (d *Drone) iniciarMissao(ocorrenciaID string, alvo models.Coordenada) {

O que faz: Transição de estados:

DISPONIVEL→DESPACHADO→EM_MISSAO→RETORNANDO→DISPONIVEL. Simula deslocamento, missão e retorno; 10% de chance de ABATIDO com encerramento do processo.

abortarMissao — code/cmd/drone/drone_main.go:283

Assinatura: func (d *Drone) abortarMissao() {

O que faz: Se em despacho/missão, muda estado para RETORNANDO e notifica broker.

notificarEstado — code/cmd/drone/drone_main.go:299

Assinatura: func (d *Drone) notificarEstado(estados models.EstadoDrone, ocID string, pos models.Coordenada) {

O que faz: Envia MensagemDrone DRONE_ESTADO com estado/posição/ocorrência para o broker (controle de escrita com connMu).

Concorrência/Redes — pontos de interesse (scan)

Categoria	Linha	Trecho
goroutine	121	go d.loopKeepalive(stopKA)
goroutine	195	go d.executarComando(cmd)
ticker	159	ticker := time.NewTicker(keepalive Intv)
mutex	30	mu sync.Mutex
mutex	34	connMu sync.Mutex
tcp_dial	90	conn, err := net.DialTimeout("tcp", addr, 5*time.Second)
json	106	d.encoder = json.NewEncoder(conn)
json	192	if err := json.Unmarshal(scanner. Bytes(), &cmd); err != nil {

code/cmd/monitor/monitor_main.go

Processo Monitor: TCP observer nos brokers + dashboard HTTP/WebSocket.

Função a função

wsAccept — code/cmd/monitor/monitor_main.go:25

Assinatura: func wsAccept(k string) string {

O que faz: Calcula Sec-WebSocket-Accept (SHA1 + base64) para handshake RFC 6455.

wsUpgrade — code/cmd/monitor/monitor_main.go:31

Assinatura: func wsUpgrade(w http.ResponseWriter, r *http.Request) (net.Conn, *bufio.ReadWriter, error) {

O que faz: Faz upgrade HTTP→WebSocket via Hijacker e responde 101 com headers; retorna net.Conn "raw".

wsFrame — code/cmd/monitor/monitor_main.go:47

Assinatura: func wsFrame(payload []byte) []byte {

O que faz: Enquadra payload em frame WebSocket (texto, FIN=1). Não mascara (servidor→cliente).

wsLer — code/cmd/monitor/monitor_main.go:62

Assinatura: func wsLer(r io.Reader) ([]byte, error) {

O que faz: Lê frame WebSocket do cliente e aplica unmask se necessário; usado apenas para detectar desconexão.

add — code/cmd/monitor/monitor_main.go:107

Assinatura: func (h *Hub) add(c *wsClient) {

O que faz: Registra cliente no hub (estrutura compartilhada com RWMutex).

remove — code/cmd/monitor/monitor_main.go:113

Assinatura: func (h *Hub) remove(c *wsClient) {

O que faz: Remove cliente do hub e fecha conexão.

broadcast — code/cmd/monitor/monitor_main.go:120

Assinatura: func (h *Hub) broadcast(data []byte) {

O que faz: Envia snapshot para todos os clientes; cada envio ocorre em goroutine e com mutex por cliente para serializar writes.

addEvento — code/cmd/monitor/monitor_main.go:164

Assinatura: func addEvento(tipo, msg, nivel string) {

O que faz: Adiciona evento de log (buffer de 100 itens) ao estado global; protegido por estadoMu.

snapshot — code/cmd/monitor/monitor_main.go:178

Assinatura: func snapshot() []byte {

O que faz: Cria cópias do estado global (drones/brokers/eventos) e serializa em JSON para WS e /api/estado.

conectarBroker — code/cmd/monitor/monitor_main.go:199

Assinatura: func conectarBroker(addr string) {

O que faz: Mantém TCP com cada broker observado (backoff), registra-se via MsgRegistro e processa stream de mensagens para atualizar UI.

processarMensagem — code/cmd/monitor/monitor_main.go:259

Assinatura: func processarMensagem(msg models.MensagemBroker, addr string) {

O que faz: Aplica MensagemBroker recebida ao estado do monitor (drones/brokers) e gera eventos para o log UI.

handleWS — code/cmd/monitor/monitor_main.go:318

Assinatura: func handleWS(w http.ResponseWriter, r *http.Request) {

O que faz: Aceita conexão WS do navegador, adiciona ao hub, envia snapshot inicial e fica lendo frames até cair.

main — code/cmd/monitor/monitor_main.go:340

Assinatura: func main() {

O que faz: Inicia goroutines de observação (uma por broker), ticker de broadcast de snapshots e ticker de timeout de brokers; expõe rotas HTTP/WS e assets.

Funções JavaScript embutidas (dashboardHTML)

Linha (no .go)	Nome	Papel (UI)
754	conectar (function)	Cria WebSocket do browser para /ws e

		registra handlers (onopen/onclose/onmessage).
765	renderTudo (function)	Pipeline de renderização: atualiza header, listas e mapa a partir do estado.
773	atualizarHeader (function)	Atualiza contadores (disponíveis/em missão/perdidos/total) e UI de status.
784	renderDrones (function)	Renderiza cards/lista de drones na coluna esquerda (estado, bateria, posição, broker).
807	renderBrokers (function)	Renderiza cards de brokers (vivo/morto, último HB) na coluna direita.
825	renderLog (function)	Renderiza log de eventos (últimos 100) com cores por severidade.
838	renderMapa (function)	Desenha mapa no canvas (posições e movimentos), quadrantes e overlays.
863	drawSec (arrow)	Helper: desenha quadrante/setor no canvas.
900	toScreen (arrow)	Helper: converte coordenadas cartesianas do modelo para pixels do canvas.

Concorrência/Redes — pontos de interesse (scan)

Categoria	Linha	Trecho
goroutine	125	go func() {
goroutine	354	go conectarBroker(addr)
goroutine	358	go func() {
goroutine	367	go func() {
ticker	359	ticker := time.NewTicker(1 * time.Second)
ticker	368	ticker := time.NewTicker(5 * time.Second)
mutex	97	mu sync.Mutex
mutex	101	mu sync.RWMutex
mutex	158	estadoMu sync.RWMutex
tcp_dial	202	conn, err := net.DialTimeout("tcp", addr, 5*time.Second)
http_server	394	if err := http.ListenAndServe(":"+* porta, nil); err != nil {
ws	21	// — WebSocket RFC 6455
ws	31	func wsUpgrade(w http.ResponseWriter, r *http.Request) (net.Conn, *bufio.ReadWriter, error)

		{
WS	40	key := r.Header.Get("Sec-WebSocket-Key")
WS	41	resp := "HTTP/1.1 101 Switching Protocols\r\nUpgrade: websocket\r\nConnection: Upgrade\r\nSec-WebSocket-Accept: " + wsAccept(key) + "\r\n\r\n"
WS	47	func wsFrame(payload []byte) []byte {
WS	93	// — Hub de clientes WebSocket
WS	121	frame := wsFrame(data)
WS	316	// — HTTP + WebSocket
WS	319	conn, _, err := wsUpgrade(w, r)
WS	753	// — WebSocket
WS	755	ws = new WebSocket('ws://' + location.host + '/ws');

json	193	data, _ := json.Marshal(estado)
json	225	json.NewEncoder(conn).E ncode(reg)
json	240	if err := json.Unmarshal(scanner. Bytes(), &msg); err != nil {

code/cmd/sensor/sensor_main.go

Processo Sensor: gera leituras e publica por UDP multicast periodicamente.

Função a função

main — code/cmd/sensor/sensor_main.go:14

Assinatura: func main() {

O que faz: Parseia flags, abre UDP para o endereço multicast e envia leituras JSON a cada intervalo usando ticker.

gerarLeitura — code/cmd/sensor/sensor_main.go:59

Assinatura: func gerarLeitura(id, tipo, setor string, x, y float64)
models.LeituraSensor {

O que faz: Gera LeituraSensor pseudo-aleatória coerente por tipo; injeta 2% de eventos críticos para estimular o despacho.

Concorrência/Redes — pontos de interesse (scan)

Categoria	Linha	Trecho
ticker	42	ticker := time.NewTicker(time.Dur ation(*intervalo) * time.Millisecond)
udp_dial	34	conn, err := net.DialUDP("udp", nil, addr)
json	47	dados, err :=

json.Marshal(leitura)

[code/generate_compose.py](#)

Gera docker-compose.yml com sensores aleatórios por setor (script de apoio).

[Concorrência/Redes — pontos de interesse \(scan\)](#)

Categoria	Linha	Trecho
-----------	-------	--------

Sem hits por scan automático.

[code/go.mod](#)

Módulo Go (Go 1.23).

[Concorrência/Redes — pontos de interesse \(scan\)](#)

Categoria	Linha	Trecho
goroutine	3	go 1.23

[code/internal/fila/fila.go](#)

Fila de prioridade thread-safe para ocorrências (heap + mutex).

[Função a função](#)

Len — code/internal/fila/fila.go:34

Assinatura: func (h heapInterno) Len() int { return len(h) }

O que faz: Implementa heap.Interface: tamanho do heap interno.

Less — code/internal/fila/fila.go:37

Assinatura: func (h heapInterno) Less(i, j int) bool {

O que faz: Define ordenação da fila: prioridade desc, Lamport asc, timestamp/FIFO asc.

Swap — code/internal/fila/fila.go:47

Assinatura: func (h heapInterno) Swap(i, j int) {

O que faz: Implementa swap e atualiza índices exigidos pelo heap.

Push — code/internal/fila/fila.go:53

Assinatura: func (h *heapInterno) Push(x interface{}) {

O que faz: Push do heap.Interface: insere item e seta índice.

Pop — code/internal/fila/fila.go:60

Assinatura: func (h *heapInterno) Pop() interface{} {

O que faz: Pop do heap.Interface: remove último item e ajusta slice.

Nova — code/internal/fila/fila.go:77

Assinatura: func Nova() *FilaPrioridade {

O que faz: Construtor: inicializa heap e retorna FilaPrioridade pronta (thread-safe via mutex).

Enfileirar — code/internal/fila/fila.go:85

Assinatura: func (fp *FilaPrioridade) Enfileirar(oc models.Ocorrencia) {

O que faz: Locka a fila, evita duplicatas por ID, e insere ocorrência no heap com prioridade inicial baseada na criticidade.

Desenfileirar — code/internal/fila/fila.go:105

Assinatura: func (fp *FilaPrioridade) Desenfileirar() (models.Ocorrencia, bool) {

O que faz: Locka a fila e remove a ocorrência de maior prioridade; usado no despacho.

Peek — code/internal/fila/fila.go:118

Assinatura: func (fp *FilaPrioridade) Peek() (models.Ocorrencia, bool) {

O que faz: Locka a fila e retorna o topo sem remover; usado para decisão de despacho sem consumir.

Remover — code/internal/fila/fila.go:129

Assinatura: func (fp *FilaPrioridade) Remover(id string) bool {

O que faz: Remove ocorrência por ID (heap.Remove) — usado quando outro broker já atendeu.

Tamanho — code/internal/fila/fila.go:143

Assinatura: func (fp *FilaPrioridade) Tamanho() int {

O que faz: Retorna o tamanho atual de pendências.

Snapshot — code/internal/fila/fila.go:151

Assinatura: func (fp *FilaPrioridade) Snapshot() []models.Ocorrencia {

O que faz: Gera cópia ordenada da fila sem destruir o heap original; pensado para replicação/sincronização.

Envelhecer — code/internal/fila/fila.go:174

Assinatura: func (fp *FilaPrioridade) Envelhecer() {

O que faz: Recalcula prioridades por envelhecimento (espera/intervalo) e re-heapifica; executado periodicamente em goroutine do broker.

Concorrência/Redes — pontos de interesse (scan)

Categoria	Linha	Trecho
mutex	72	mu sync.Mutex

code/internal/models/dados.go

Modelos de dados e mensagens (sensores, ocorrências, drones, comandos).

Função a função

String — code/internal/models/dados.go:14

Assinatura: func (c Criticidade) String() string {

O que faz: Converte o enum Criticidade para texto (BAIXA/ALTA/DESCONHECIDA); usado em logs/UI.

Distancia — code/internal/models/dados.go:34

Assinatura: func (c Coordenada) Distancia(outro Coordenada) float64 {

O que faz: Calcula distância euclidiana ao quadrado (dx^2+dy^2) para comparação de proximidade sem sqrt; usado no algoritmo de escolha do drone.

Disponivel — code/internal/models/dados.go:96

Assinatura: func (d *InfoDrone) Disponivel() bool {

O que faz: Retorna se o drone está apto a ser despachado (estado DISPONIVEL e bateria >10%).

Concorrência/Redes — pontos de interesse (scan)

Categoria	Linha	Trecho
-----------	-------	--------

Sem hits por scan automático.

[docker-compose.yml](#)

Orquestração do cluster: 4 brokers, 6 drones, 1 monitor e sensores (containers).

Concorrência/Redes — pontos de interesse (scan)

Categoria	Linha	Trecho
-----------	-------	--------

Sem hits por scan automático.

[docs/HormuzNet_Analise_Concorrencia_Conectividade.docx](#)

Arquivo binário/asset.

[docs/arquitetura_descentralizada.md](#)

Documento de arquitetura (Lamport, P2P, UDP multicast, exclusão mútua).

Concorrência/Redes — pontos de interesse (scan)

Categoria	Linha	Trecho
-----------	-------	--------

ws	33	A camada de visão atua como uma interface de consolidação. Projetada como um painel *web*, ela recebe dados via WebSockets de toda a rede de Brokers. O design da interface exibe o Estreito de Ormuz mapeado sob quatro qua
----	----	--

ws	83	B1 ==> WebSockets M
----	----	-----------------------

ws	84	B4 ==> WebSockets M
----	----	-----------------------

start.sh

Script para rodar localmente (go run) em múltiplos terminais; útil fora do Docker.

Função a função

open_term — start.sh:28

Assinatura: open_term() {

O que faz: Descrição não encontrada no mapeamento manual (revisar caso seja função crítica).

start_brokers — start.sh:60

Assinatura: start_brokers() {

O que faz: Descrição não encontrada no mapeamento manual (revisar caso seja função crítica).

start_drones — start.sh:85

Assinatura: start_drones() {

O que faz: Descrição não encontrada no mapeamento manual (revisar caso seja função crítica).

start_sensores — start.sh:106

Assinatura: start_sensores() {

O que faz: Descrição não encontrada no mapeamento manual (revisar caso seja função crítica).

Concorrência/Redes — pontos de interesse (scan)

Categoria	Linha	Trecho
goroutine	12	# Pré-requisito: go build ./... já executado
goroutine	63	"cd '\$ROOT' && go run ./cmd/broker/ \
goroutine	74	"cd '\$ROOT' && go run ./cmd/broker/ \

goroutine	88	"cd '\$ROOT' && go run ./cmd/drone/ \
goroutine	97	"cd '\$ROOT' && go run ./cmd/drone/ \
goroutine	110	"cd '\$ROOT' && go run ./cmd/sensor/ \
goroutine	117	"cd '\$ROOT' && go run ./cmd/sensor/ \

stop.sh

Script para encerrar processos iniciados via go run/binários locais.

Concorrência/Redes — pontos de interesse (scan)

Categoria	Linha	Trecho
goroutine	12	PIDS=\$(pgrep -f "go run ./cmd/\${PROC %_main}"/" 2>/dev/null true)

5. Concorrência — explicação integrada

Concorrência é gerada de forma explícita via goroutines (Go) e implicitamente via múltiplos processos/containers (Docker Compose). No broker, há paralelismo por: (1) datagrama UDP (processarLeitura em goroutine), (2) conexão TCP aceita (identificarConexao em goroutine), e (3) loops periódicos (tickers) para heartbeat, detecção de falhas, envelhecimento de fila e despacho. Nos drones, há paralelismo por keepalive (goroutine) e por comando recebido (goroutine por comando). No monitor, há paralelismo por broker observado (uma goroutine por conexão) e por broadcast periódico via WebSocket.

- code/cmd/broker/broker_main.go:114 — Goroutines principais do broker (TCP/UDP/loops).
- code/cmd/broker/broker_main.go:148 — Goroutine por leitura UDP (paralelismo por evento).
- code/cmd/broker/broker_main.go:197 — Goroutine por conexão TCP aceita.
- code/cmd/drone/drone_main.go:121 — Keepalive em goroutine separada.
- code/cmd/drone/drone_main.go:195 — Execução concorrente por comando (potencial interleaving).
- code/cmd/monitor/monitor_main.go:354 — Monitor observa brokers em paralelo.
- code/cmd/monitor/monitor_main.go:125 — Goroutines periódicas do monitor (broadcast/timeout).

5.1 Problemas potenciais (race/deadlock/starvation) e tratamento atual

- Race conditions locais: mitigadas com mutex/RWMutex nos mapas compartilhados (drones, vizinhos, atendidos, estado do monitor).
- Deadlocks: não há locks aninhados complexos, mas há regiões onde múltiplos mutex podem ser tomados em sequência (ex: processarMensagemDrone manipula dronesMu e em seguida chama broadcastVizinhos). A chance de deadlock é baixa, porém o modelo depende da disciplina de lock ordering.
- Starvation: a fila tenta evitar starvation com envelhecimento (Envelhecer) — porém o teto atual limita ao nível de criticidade ALTA, então “envelhecimento” não cria novos níveis além de ALTA.

- Exclusão mútua distribuída: abordagem simplificada (broker com conexão do drone) pode falhar sob inconsistências temporárias de estado, mas reduz bastante a chance de “duplo despacho”.
- Backpressure: `broadcastVizinhos` e `hub.broadcast` criam goroutines por destinatário; sob carga/links lentos, pode haver explosão de goroutines. Produção exigiria filas/buffers e workers.

6. Protocolos e conectividade

Resumo por camada:

- Sensores → Brokers: UDP multicast (224.0.0.1:8080), payload JSON (LeituraSensor).
- Drone ↔ Broker: TCP, mensagens JSON por linha (MensagemDrone/ComandoDrone).
- Broker ↔ Broker: TCP, mensagens JSON por linha (MensagemBroker) + heartbeat + Lamport.
- Monitor ↔ Brokers: TCP observer (MensagemBroker).
- Navegador ↔ Monitor: HTTP (dashboard e /api/estado) + WebSocket (/ws) para push de snapshots.

6.1 Onde aparece no código (refs rápidas)

- code/cmd/broker/broker_main.go:135 — Broker recebe multicast UDP.
- code/cmd/sensor/sensor_main.go:34 — Sensor envia UDP.
- code/cmd/broker/broker_main.go:185 — Broker aceita TCP.
- code/cmd/drone/drone_main.go:90 — Drone conecta TCP (fallback).
- code/cmd/monitor/monitor_main.go:394 — Monitor expõe HTTP.
- code/cmd/monitor/monitor_main.go:40 — Handshake WebSocket.

7. Pontos de atenção para defesa (perguntas prováveis)

Checklist de respostas rápidas (com gancho no código):

Pergunta: Concorrência entre drones

Resposta preparada: Cada drone é um processo (docker-compose: serviço drone1..drone6). Dentro do broker, tentarDespachar decide e envia comandos. Ver: broker_main.go:tentarDespachar e docker-compose.yml serviços drone*.

Pergunta: Exclusão mútua

Resposta preparada: Só o broker que tem a conexão TCP do drone escolhido envia o comando; depois faz broadcast DRONE_DESPACHADO para outros brokers removerem a ocorrência.

Pergunta: Failover do drone

Resposta preparada: loopConexao tenta brokers em ordem (flag -brokers) com backoff; ao perder conexão, tenta o próximo.

Pergunta: Detecção de falhas entre brokers

Resposta preparada: loopHeartbeat + loopDetectarFalhas, baseado em timeouts.

Pergunta: Por que multicast

Resposta preparada: Um sensor publica uma vez; todos os brokers recebem simultaneamente → redundância e menor overhead.

Pergunta: Por que WebSocket manual

Resposta preparada: Didático: handshake e framing explícitos; porém limitado (não implementa ping/pong/close completo).